SISTEMA DE TELEMETRIA PARA MONITORAMENTO DE FLUXO DE ÁGUA UTILIZANDO MICROCONTROLADORES E COMUNICAÇÃO WI-FI

Bernardo Goulart Búrigo[1], Matheus Leandro Ferreira[2]

Resumo: A medição por hidrômetro geral, adotada na maioria dos condomínios, não oferece aos gestores visibilidade granular e imediata sobre o consumo de água, em um país que perde 37,78% da água tratada na distribuição. Este trabalho propõe um sistema de telemetria baseado em Internet das Coisas (IoT) para o monitoramento contínuo do consumo por unidade habitacional, com hardware de baixo custo e software de código aberto. O protótipo utiliza o sensor de fluxo YF-S201 e o microcontrolador ESP32, que converte os pulsos em vazão e volume e os envia por Wi-Fi, em requisições HTTPS com JSON, a uma API REST em Spring Boot. As leituras são persistidas como série temporal em PostgreSQL com TimescaleDB e exibidas em uma aplicação web e móvel em React Native com Expo, com controle de acesso por perfil de síndico e de proprietário e alerta automático de fluxo contínuo, indicativo de vazamento. A validação experimental, em bancada, com faixas de vazão baixa, média e alta e repetições, comparou as leituras com volumes de referência. Após a calibração, o erro percentual médio foi de X,XX%, a latência média de atualização de X,X s e a taxa de entrega de XX,X% em rede estável, o que demonstra a viabilidade técnica da solução para a gestão hídrica em condomínios.

Palavras-chave: internet das coisas; telemetria; monitoramento de água; condomínios; ESP32; Spring Boot.

ABSTRACT: Single-meter measurement, adopted in most condominiums, gives managers no granular or immediate visibility over water consumption, in a country that loses 37.78% of its treated water in distribution. This work proposes an Internet of Things (IoT) telemetry system for the continuous monitoring of consumption per housing unit, using low-cost hardware and open-source software. The prototype uses the YF-S201 flow sensor and the

[1]Curso de Ciência da Computação, Universidade do Extremo Sul Catarinense (Unesc), Criciúma - Santa Catarina - Brasil. bernardoburigo@unesc.net

[2]Orientador(a), Curso de Ciência da Computação, Universidade do Extremo Sul Catarinense (Unesc), Criciúma - Santa Catarina - Brasil. mlf@unesc.net

ESP32 microcontroller, which converts pulses into flow rate and volume and sends them over Wi-Fi, through HTTP requests with JSON, to a REST API built with Spring Boot. Readings are stored as a time series in PostgreSQL with TimescaleDB and displayed in a web and mobile application built with React Native and Expo, with role-based access for building managers and unit owners and automatic alerts for continuous flow, a sign of leaks. The experimental validation, on a test bench with low, medium and high flow ranges and repetitions, compared the readings with reference volumes. After calibration, the mean percentage error was X.XX%, the mean update latency was X.X s and the delivery rate was XX.X% on a stable network, demonstrating the technical feasibility of the solution for water management in condominiums.

Keywords: internet of things; telemetry; water monitoring; condominiums; ESP32; Spring Boot.

1 INTRODUÇÃO

A relevância do monitoramento inteligente de recursos naturais tem crescido expressivamente no contexto urbano e nas infraestruturas modernas, e a Internet das Coisas (IoT) vem se consolidando como ferramenta estratégica para a gestão eficiente desses recursos. O termo, cunhado por Ashton (2009) para descrever a conexão de objetos do cotidiano à internet por meio de sensores e identificadores, evoluiu para uma infraestrutura global e dinâmica na qual objetos físicos e virtuais se interconectam por protocolos padronizados e interoperáveis (Magrani, 2018).

O avanço dessa tecnologia pode ser observado no mercado brasileiro, que encerrou o ano de 2025 com 53,7 milhões de acessos IoT, um crescimento de 13,6% em relação ao ano anterior, evidenciando a expansão do uso de dispositivos conectados para coleta e análise de dados em tempo real (TELETIME, 2026).

Dentro desse quadro, a administração hídrica destaca-se como uma das áreas mais críticas. Dados indicam que 37,78% da água tratada no Brasil é perdida na distribuição antes mesmo de chegar às residências, o que evidencia falhas estruturais e operacionais significativas (SANEAMENTO HOJE, 2024).

Relatórios internacionais reforçam que o monitoramento contínuo e o compartilhamento de dados são essenciais para a segurança hídrica global e para a prosperidade das comunidades urbanas (UNESCO,

2024), e que a gestão sustentável da água constitui um pilar fundamental para a paz e a prosperidade, exigindo inovações tecnológicas para garantir a disponibilidade do recurso (UNESCO, 2025).

Em condomínios residenciais, a limitação dos métodos tradicionais de controle contribui diretamente para esse cenário de ineficiência. A medição costuma ser realizada de forma manual e esporádica, com base na leitura de um hidrômetro geral que registra apenas o consumo total da edificação. Esse modelo não fornece dados sobre o perfil de uso diário ou por unidade, dificultando a identificação rápida de anomalias no emprego do recurso hídrico (Barros et al., 2023). A ausência de um histórico detalhado torna o gerenciamento reativo: uma irregularidade, como um vazamento interno, só se torna perceptível quando já se refletiu na fatura mensal.

O problema central deste trabalho é, portanto, a dificuldade de gestores condominiais obterem visibilidade granular e imediata sobre o consumo de água, o que motiva a questão de pesquisa: como a integração entre sensores de fluxo, microcontroladores e redes de comunicação Wi-Fi pode viabilizar o monitoramento individualizado e contínuo do consumo de água em condomínios residenciais?

A utilização de sensores de fluxo integrados a microcontroladores de baixo custo permite a medição da vazão de água diretamente na infraestrutura hidráulica e, quando esses dispositivos são conectados a uma arquitetura IoT, os dados podem ser processados e transmitidos por redes sem fio, ampliando a visibilidade sobre o uso e permitindo a detecção rápida de desvios (Silva, 2024b).

A gestão inteligente de água (*Smart Water*) aplica exatamente esse conjunto de tecnologias para monitorar, controlar e otimizar o consumo, substituindo estimativas periódicas por leituras em tempo real e transformando uma gestão passiva em um controle proativo e baseado em dados (Silva et al., 2025).

O ESP32, desenvolvido pela Espressif Systems, destaca-se nesse contexto por integrar em um único chip um processador de dois núcleos, conectividade Wi-Fi e Bluetooth e baixo consumo de energia, o que o torna uma escolha adequada para aplicações embarcadas de baixo custo (Santos et al., 2016). Sua capacidade de realizar leituras frequentes de sensores de fluxo e transmiti-las de forma estável o credencia, ainda, especificamente para projetos de monitoramento hídrico (Silva, 2024a).

Com o objetivo de compreender o estado da arte relacionado ao monitoramento hídrico baseado em Internet das Coisas, foram analisadas

pesquisas correlatas que empregam sensores, microcontroladores, comunicação sem fio e técnicas computacionais para aquisição e análise de dados de consumo e vazão. Rolle (2024) propôs um sistema de detecção e localização de vazamentos em redes de distribuição utilizando sensores IoT integrados a algoritmos de aprendizado de máquina, com alta precisão na localização de falhas, porém dependente de infraestrutura computacional robusta e manutenção constante dos modelos.

Silva (2024b) desenvolveu um sistema de monitoramento de pressões e vazões em nível municipal, apoiado em inteligência artificial para processar grandes volumes de dados, eficiente para redes macro, mas de difícil acolhimento em microambientes como condomínios. Barros et al. (2023) aplicaram sensores IoT à medição simultânea de água e energia em um laboratório, validando a confiabilidade de sensores de baixo custo em condições ideais, nas quais as variáveis de uso são controladas e previsíveis.

Mais próximos do escopo residencial, Medeiros et al. (2021) desenvolveram um protótipo para detecção de vazamentos em edificações com um sensor turbina acoplado a um NodeMCU com Wi-Fi e alimentação solar, identificando perdas de aproximadamente 12,81 litros em um único dia; a solução, contudo, não contemplava o monitoramento individualizado por unidade nem uma interface de visualização para os moradores. Mascarenhas (2024) construiu um medidor inteligente com Arduino, sensor de vazão e módulo ESP8266, registrando os dados em planilhas do Google e enviando alertas por e-mail, mas relatou limitações de autonomia energética e de estabilidade da comunicação, além da ausência de uma aplicação dedicada.

Observa-se, assim, uma lacuna entre soluções de larga escala, dependentes de inteligência artificial, e protótipos residenciais que carecem de individualização por unidade, de uma aplicação própria com controle de acesso e de uma validação quantitativa que mensure não apenas a precisão do sensor, mas também a latência e a confiabilidade da transmissão.

Diante dessa lacuna, este trabalho propõe o desenvolvimento de um sistema de telemetria baseado em sensores de fluxo integrados a microcontroladores, responsáveis pela coleta contínua dos dados de consumo de água e por sua transmissão por comunicação sem fio a uma Interface de Programação de Aplicações (API, do inglês *Application Programming Interface*), que armazena as leituras e as disponibiliza em um painel de visualização.

O objetivo geral é propor um modelo de gestão hídrica em condomínios por meio do monitoramento contínuo do consumo de água. Para alcançá-lo, foram definidos os seguintes objetivos específicos: Modelar a arquitetura de um sistema de monitoramento de fluxo de água baseado em sensores e microcontroladores; Desenvolver um protótipo de coleta de dados utilizando sensor de fluxo, microcontrolador e comunicação sem fio; Implementar uma API e um painel de visualização para armazenamento e monitoramento das leituras; Validar o funcionamento do sistema por meio de testes de bancada, comparando as leituras do protótipo com um medidor de referência para mensurar a margem de erro e a precisão da coleta de dados.

Sob o viés acadêmico, o trabalho integra sistemas embarcados, redes de comunicação sem fio e arquitetura de sistemas distribuídos em um protótipo funcional, consolidando conhecimento prático e científico (Silva, 2021). Do ponto de vista social, a abordagem pode contribuir para a redução do desperdício, a diminuição de custos operacionais e o apoio à tomada de decisão por síndicos e gestores, ao tornar o monitoramento mais eficiente e acessível (Silva et al., 2025).

O restante deste artigo está organizado da seguinte forma: a Seção 2 descreve os materiais e métodos, incluindo a arquitetura, o hardware, o firmware, a API, o banco de dados, a aplicação e o procedimento experimental; a Seção 3 apresenta e discute os resultados obtidos e os compara com os trabalhos correlatos; e a Seção 4 traz as conclusões, limitações e trabalhos futuros.

2 MATERIAIS E MÉTODOS

Quanto à natureza, a pesquisa é aplicada, voltada ao desenvolvimento de um protótipo funcional para um problema prático de gestão hídrica em condomínios; quanto à abordagem, é predominantemente quantitativa; e, quanto aos objetivos, é exploratória e descritiva, por investigar a viabilidade técnica da solução e detalhar seu funcionamento.

Quanto aos procedimentos, a pesquisa caracteriza-se como experimental, uma vez que o protótipo foi submetido a condições controladas de vazão e volume para mensuração de seu erro e desempenho. A variável independente é a faixa de vazão aplicada ao sensor, e as variáveis dependentes são o erro de medição, a dispersão das leituras, a taxa de entrega das mensagens e a latência de atualização, detalhadas na Seção 2.6.

O desenvolvimento foi incremental, em oito etapas (*sprints* 0 a 7)

executadas entre agosto e dezembro de 2026, cada uma com um critério objetivo de validação, e todo o código foi mantido em um único repositório com os diretórios **firmware**, **backend**, **app** e **docs** e um arquivo Docker Compose que reproduz o ambiente de execução.

2.1 ARQUITETURA DO SISTEMA

A arquitetura segue o modelo de três camadas comumente adotado na literatura de IoT: percepção, rede e aplicação (Rodrigues et al., 2014). A camada de percepção é composta pelo sensor de fluxo YF-S201 e pelo microcontrolador ESP32; a camada de rede, pela rede Wi-Fi local e pelo protocolo HTTPS; e a camada de aplicação, pela API REST em Spring Boot, pelo banco PostgreSQL com TimescaleDB e pelo painel em React Native com Expo, conforme a Figura 1. O modelo de domínio foi concebido para múltiplas unidades: um **Usuario** tem o papel de síndico ou de proprietário, cada **Condominio** agrega vários **Apartamento**, cada apartamento possui um ou mais **Sensor**, e cada sensor produz uma sequência de **Leitura**.

Figura 1 - Arquitetura geral do sistema

Figura: [YF-S201] –pulsos– [ESP32] –HTTPS POST (JSON) via Wi-Fi– [Túnel Cloudflare] – [API Spring Boot + PostgreSQL/TimescaleDB no servidor doméstico] – [App web na Vercel / app móvel (Expo)] via polling

Fonte: Do autor.

2.2 HARDWARE E FIRMWARE

O microcontrolador utilizado foi uma placa ESP32 DevKit V1, escolhida pelo Wi-Fi nativo, pelo processador de dois núcleos e pelo baixo custo (Santos et al., 2016; ESPRESSIF SYSTEMS, 2026), alimentado pela sua entrada Micro-USB. O sensor YF-S201 baseia-se no efeito Hall: a passagem da água gira o rotor, e o sensor Hall gera pulsos com frequência proporcional à vazão, capturados por uma entrada digital. Segundo o fabricante (SEEED STUDIO, 2026), o sensor opera de 1 a 30 L/min, com rosca de 1/2 polegada, e a vazão e o volume são obtidos pela Equação 1:

$$Q = \frac{F}{K}, \qquad V = \frac{P_{total}}{K \times 60} \tag{1}$$

em que Q é a vazão (L/min), F a frequência dos pulsos (Hz), V o volume (L), P_{total} o total de pulsos no período e K o fator de calibração, de valor teórico 7,5, o que equivale a 450 pulsos por litro.

A folha de dados especifica alimentação de 5 a 18 V e, com 5 V,

um nível alto de saída de ao menos 4,6 V, acima dos 3,3 V aceitos pelas entradas do ESP32. Por isso, o sensor foi alimentado com os 3,3 V da própria placa, abaixo da faixa especificada, o que mantém o sinal compatível com a entrada sem divisor de tensão; como o fator K foi calibrado nessa condição, o efeito da tensão está incorporado à calibração.

O firmware foi desenvolvido em C++ com o PlatformIO (PLATFORMIO, 2026), em dois ambientes: **esp32dev**, para o hardware, e **native**, que executa testes unitários no computador. A contagem de pulsos usa interrupção externa (**attachInterrupt**), de modo que nenhum pulso seja perdido, e a conversão de pulsos, a montagem do JSON e a fila de reenvio foram isoladas em bibliotecas próprias (**FlowMeter**, **LeituraPayload** e **SendQueue**), sem dependência do núcleo Arduino, testadas com o *framework* Unity antes de o hardware estar disponível.

A rede é configurada pela biblioteca WiFiManager, por um portal cativo, sem recompilação. A constante de calibração tem um valor padrão no código, e o valor efetivo de cada sensor é gravado na memória não volátil do ESP32 e editado pelo mesmo portal, para que cada sensor receba o seu após a calibração (Seção 2.6.2). O relógio é sincronizado por NTP, para que cada leitura carregue o instante de captura gerado no dispositivo. A cada 5 segundos, o firmware envia por HTTP POST um JSON com o identificador do sensor, o instante, os pulsos, o volume estimado e um contador incremental (**sequencia**), autenticando-se pelo cabeçalho **X-Sensor-Token**. Cada leitura entra em uma fila em memória de 200 posições (cerca de 16 minutos de operação) e é enviada em ordem; em caso de falha de rede ou do servidor, é reenviada com intervalo crescente, de 5 a 60 s, e a resposta 409 a um reenvio já persistido conta como confirmação. Leituras só se perdem se a fila se esgotar ou se o ESP32 reiniciar com envios pendentes, o que aparece como lacuna na numeração recebida pelo servidor.

Optou-se pelo HTTP no estilo REST, comum em APIs para IoT pela simplicidade (Pavaneli, 2023; RED HAT, 2023), em vez de MQTT, porque o fluxo é unidirecional, do sensor para o servidor: o HTTP dispensa um *broker*, é nativo no ESP32 e devolve códigos de estado, como a rejeição de duplicatas, ao custo de maior sobrecarga por mensagem, aceitável no intervalo adotado.

2.3 API E BACKEND

A API foi desenvolvida em Java 21 com Spring Boot 3 (SPRING, 2026), em camadas de controladores REST, serviços e repositórios Spring

Data JPA, com DTOs separados das entidades. Os cadastros usam exclusão lógica, o esquema é versionado pelo Flyway em oito migrações, a documentação é gerada pelo springdoc-openapi (Swagger UI) e o Spring Actuator fornece verificações de integridade. A Tabela 1 resume os principais endpoints.

Tabela 1 - Principais endpoints da API

Método e rota	**Autenticação**	**Função**
POST /api/auth/registrar	X-Admin-Token	Cadastro de usuário, com chave administrativa
POST /api/auth/login	Pública	Emissão de token JWT
CRUD /api/condominios, /api/apartamentos, /api/ sensores	JWT (síndico)	Cadastros, com exclusão lógica
POST /api/leituras	X-Sensor-Token	Ingestão; rejeita duplicatas (409)
GET /api/apartamentos/ {id}/consumo	JWT (síndico ou proprietário)	Consumo por período, por hora ou dia

Fonte: Do autor.

A segurança, com Spring Security, combina dois mecanismos: usuários autenticam-se por login e recebem um token JWT, sendo o cadastro protegido por uma chave administrativa, pois a API fica exposta à internet, e dispositivos usam uma chave de API gerada na criação de cada sensor, o que dispensa login no microcontrolador e permite revogar um dispositivo isoladamente. Os cadastros exigem o papel de síndico, e o consumo de um apartamento só pode ser consultado pelo síndico do condomínio ou pelo proprietário da unidade; qualquer outro acesso, inclusive de síndico de outro condomínio, é negado com o código 403. Na ingestão, uma leitura com mesmo sensor, sequência e instante é recusada com o código 409, o que impede que reenvios inflem o volume; o token inválido recebe 401, e o payload malformado, rejeitado por Bean Validation, 400.

Sobre as leituras persistidas, o serviço **DetectorFluxoContinuo** identifica o principal sintoma de vazamento: quando um sensor registra vazão ininterrupta por mais tempo que o limite configurado (**app.alerta. fluxoContinuoMinutos**, de 30 minutos), tolerando lacunas de até 15 s entre leituras consecutivas, a API sinaliza, a cada consulta, um alerta associado ao apartamento, sujeito às mesmas regras de acesso do consumo.

2.4 BANCO DE DADOS

A persistência usa PostgreSQL 16 com a extensão TimescaleDB (TIMESCALE, 2026), em contêiner Docker, com chaves estrangeiras que encadeiam usuário, condomínio, apartamento, sensor e leitura. A tabela de leituras é uma *hypertable* particionada pelo instante de captura, de modo que consultas por período percorrem apenas as partições (*chunks*) relevantes, e a restrição de unicidade inclui essa coluna, como exige a extensão. Cada leitura guarda o sensor, o instante de captura, os pulsos, o volume estimado, a sequência e o instante de persistência, que sustentam o cálculo do consumo, da taxa de entrega e da latência. O consumo por hora e por dia é agregado sob demanda no próprio banco.

A extensão foi escolhida por tratar as leituras como série temporal sem perder a compatibilidade com o PostgreSQL, o JPA e o Flyway, e por oferecer retenção, compressão e agregados contínuos. Com envio a cada 5 s, um condomínio de 100 unidades geraria cerca de 1,7 milhão de leituras por dia, escala em que esses recursos se tornam necessários; no protótipo, nenhum deles foi configurado. O backend e o banco foram executados em um servidor doméstico (computador Lenovo ThinkCentre com Ubuntu Server), com os contêineres do Docker Compose, exposto à internet por um túnel da Cloudflare, que fornece HTTPS sem abrir portas no roteador (CLOUDFLARE, 2026); a versão web da aplicação foi publicada na Vercel (VERCEL, 2026), com o DNS de ambos gerido pela Cloudflare.

2.5 APLICAÇÃO

A aplicação foi desenvolvida em TypeScript com React Native e Expo (EXPO, 2026), o que permitiu gerar a versão web e o aplicativo móvel a partir de uma única base de código. Ela contempla telas de autenticação, de listagem de apartamentos (do condomínio, para o síndico, e da própria unidade, para o proprietário), de consumo por hora e por dia, com seleção de período, de detalhe do sensor e de alertas de fluxo contínuo, ilustradas na Figura 2. O token JWT é anexado às requisições por um interceptador Axios e armazenado com expo-secure-store no aplicativo móvel e no localStorage na versão web; as permissões não dependem da interface, pois são aplicadas pela API. A atualização é feita por *polling* com TanStack Query, a cada 5 s na tela do sensor e nos alertas e a cada 10 s nas listas e gráficos de consumo, escolhido por dispensar a infraestrutura de WebSocket ou *Server-Sent Events* (SSE). Para a medição da latência, a tela do sensor, ativada por uma variável de ambiente, registra o instante em que

cada leitura nova aparece e o exporta em CSV.

Figura 2 - Telas da aplicação: (a) painel do síndico com múltiplos apartamentos; (b) consumo de um apartamento; (c) visão do proprietário no aplicativo móvel

Três capturas lado a lado (web do síndico, gráfico com dados reais, app do proprietário).

Fonte: Do autor.

2.6 PROCEDIMENTO EXPERIMENTAL

A validação foi estruturada em quatro dimensões: precisão volumétrica, repetibilidade, comunicação e atualização. Todos os resultados de precisão foram obtidos com o sensor físico; como o protótipo dispunha de um conjunto ESP32 e YF-S201, o comportamento com múltiplas unidades foi verificado com um simulador em Python que envia leituras de sensores virtuais ao mesmo endpoint de ingestão, identificados no cadastro como simulados e usados apenas para avaliar o isolamento entre apartamentos, o painel e o alerta, este com um fluxo constante prolongado.

2.6.1 Bancada, cenários e repetições

A bancada, apresentada na Figura 3, foi montada com descrever: reservatório/torneira, registro para variação da vazão, mangueira de 1/2 polegada, sensor em trecho retilíneo. O volume de referência foi aferido em um recipiente graduado de X L e menor divisão Y mL, com incerteza de metade da menor divisão, e o tempo de escoamento, com cronômetro. Antes de cada série, a tubulação foi purgada para eliminar bolhas de ar. Em cada ensaio, o registro foi aberto até a marca de referência e fechado; o volume lido foi calculado pela diferença do contador de pulsos acumulados, exibido no monitor serial, entre o início e o fim do ensaio, para isolar o erro do sensor do erro de transmissão, e a vazão efetiva, pela razão entre volume de referência e tempo.

Figura 3 - Bancada de testes

Foto da montagem: ESP32 na protoboard, YF-S201 em linha com a mangueira, registro, recipiente graduado.

Fonte: Do autor.

A vazão foi variada em três faixas, em ordem crescente e dentro da faixa operacional do sensor, cada uma obtida com uma abertura fixa e marcada do registro, com volume de referência constante, de modo que a

vazão fosse a única variável alterada. Cada faixa foi repetida em duas campanhas independentes: a de calibração, que determinou o fator K, e a de validação, executada depois com o fator ajustado, para que o erro residual não fosse avaliado sobre as mesmas medidas que o produziram (Tabela 2). Ensaios com falha operacional evidente foram refeitos, e nenhum resultado foi removido por critério estatístico.

Tabela 2 - Delineamento experimental por faixa de vazão

Faixa	**Vazão alvo (L/min)**	**Vol. de referência (L)**	**Rep. de calibração**	**Rep. de validação**
Baixa	1,5 a 4	V	10	30
Média	6 a 12	V	10	30
Alta	15 a 25	V	10	30
Total			30	90

Fonte: Do autor.

2.6.2 Calibração do sensor

Sensores de baixo custo variam entre unidades, e as condições de instalação alteram a relação entre pulsos e volume: Mascarenhas (2024) precisou introduzir um fator de correção, e Medeiros et al. (2021) ressaltam a dependência da precisão em relação à instalação hidráulica. Por isso, $K = 7{,}5$ foi tratado como ponto de partida e ajustado, por aferição volumétrica direta sobre todas as repetições de calibração, pela Equação 2, em que $V_{lido,i}$ é o volume calculado pelo sistema e $V_{real,i}$ o volume aferido no ensaio i:

$$K_{novo} = K_{atual} \times \frac{\sum V_{lido,i}}{\sum V_{real,i}} \tag{2}$$

Na campanha de validação, o erro foi calculado sobre os mesmos pulsos com o fator teórico e com o calibrado, o que quantifica o efeito da calibração sobre dados independentes.

2.6.3 Métricas e critérios de aceitação

A precisão volumétrica foi avaliada, por faixa, pelo erro absoluto $EA_i = |V_{lido,i} - V_{real,i}|$, pelo erro percentual com sinal (e_i), cuja média é o viés, e pelo erro percentual médio (EPM), conforme a Equação 3; o efeito da calibração, pela redução relativa $R = (EPM_{antes} - EPM_{depois})/EPM_{antes} \times 100$. A repetibilidade foi avaliada pelo coeficiente de variação (CV), razão entre o desvio padrão amostral s e a média $\overline{V}_{lido}$ das n repetições de vali-

dação:

$$e_i = \frac{V_{lido,i} - V_{real,i}}{V_{real,i}} \times 100, \qquad EPM = \frac{1}{n}\sum_{i=1}^{n} |e_i|, \qquad CV = \frac{s}{\overline{V}_{lido}} \times 100 \quad (3)$$

A comunicação foi avaliada pela taxa de entrega (TE) da Equação 4, em que, para cada ciclo de funcionamento c do ESP32, já que a sequência reinicia a cada reinício, $N_{rec,c}$ são as mensagens recebidas, sem contar duplicatas rejeitadas, e $N_{esp,c}$ as esperadas, obtidas pela primeira e pela última sequência registradas no monitor serial. Como o firmware reenvia as leituras não confirmadas, também foram contadas as entregues com atraso superior a 10 s, que aguardaram na fila, e o maior atraso observado. A TE foi medida em rede estável, durante T minutos, e com falhas induzidas pelo desligamento do roteador por X s em N ocasiões.

$$TE = \frac{\sum_c N_{rec,c}}{\sum_c N_{esp,c}} \times 100, \qquad N_{esp,c} = S_{ult,c} - S_{prim,c} + 1 \quad (4)$$

A atualização foi avaliada pela latência, decomposta pelos instantes de captura (t_{cap}), gerado no ESP32, de persistência (t_{pers}), atribuído pela API, e de exibição (t_{exib}), registrado pelo painel, conforme a Equação 5, com média, mediana, percentil 95, máximo e desvio padrão em N leituras, em rede estável. Os três relógios foram sincronizados por NTP, e o desvio residual foi considerado desprezível frente a latências da ordem de segundos. Como t_{cap} corresponde ao fim da janela de 5 s, o consumo ocorrido no início da janela tem um atraso adicional de até 5 s em relação a L_{tot}.

$$L_{ing} = t_{pers} - t_{cap}, \qquad L_{exib} = t_{exib} - t_{pers}, \qquad L_{tot} = t_{exib} - t_{cap} \quad (5)$$

Os critérios de aceitação foram definidos antes dos ensaios: EPM de até 10% por faixa após a calibração, igual à precisão nominal do fabricante; CV de até 5%; TE de ao menos 95% em rede estável; e L_{tot} média de até X s, justificar: adequados ao acompanhamento do consumo em escala de minutos. Os dados foram exportados em CSV e processados em Python com pandas, por scripts de calibração, de entrega e de latência disponíveis no diretório **docs/scripts/analise** do repositório.

3 RESULTADOS E DISCUSSÃO

3.1 VERIFICAÇÃO DO SISTEMA INTEGRADO

No firmware, os 23 testes unitários das bibliotecas **FlowMeter**, **LeituraPayload** e **SendQueue** passaram no ambiente **native**, incluindo a conversão do fabricante (450 pulsos por litro), e a compilação para o **esp32dev** ocupou 80,4% da memória flash e 16,6% da RAM. Na API, um teste de integração com Testcontainers exercitou o fluxo completo contra um TimescaleDB real, do registro do síndico à consulta de consumo, incluindo a rejeição de leitura duplicada (409), de token de sensor inválido (401) e de consulta por síndico de outro condomínio (403); o mesmo fluxo foi repetido com **curl** contra os contêineres do Docker Compose, com as oito migrações aplicadas automaticamente. Um segundo teste de integração (**AlertaIntegrationTest**) confirmou que o fluxo contínuo acima do limite é sinalizado como alerta e que ele só é visível ao síndico e ao proprietário da unidade.

Com o sensor físico, ao abrir o registro, a leitura foi persistida em X s e exibida na atualização seguinte do *polling*. O painel exibiu N apartamentos, um com o sensor real e os demais com sensores virtuais, com consumos isolados: o síndico visualizou todas as unidades, cada proprietário apenas a sua, e os acessos cruzados foram negados pela API (Figura 2). Com o simulador mantendo vazão constante por mais de 30 minutos, o alerta foi exibido na aplicação X s após o limite.

Os principais obstáculos surgiram apenas na execução real, como a incompatibilidade de versão da biblioteca jjwt e uma **LazyInitializationException** do Hibernate que se manifestava como um enganoso erro 403. Nenhum deles seria detectado com o banco ou o HTTP simulados, o que justifica os testes de integração com dependências reais.

3.2 CALIBRAÇÃO DO SENSOR

A Tabela 3 apresenta os 30 ensaios de calibração com o fator teórico. O volume lido superestimou / subestimou o de referência em X%, Y% e Z% nas faixas baixa, média e alta, e a Equação 2 resultou em $K =$ X,XX (XXX pulsos por litro), X% acima / abaixo do valor teórico, o que confirma a necessidade de calibração individual.

Tabela 3 - Campanha de calibração com o fator teórico ($K = 7{,}5$)

Faixa	n	Vazão média (L/min)	Pulsos médios	Freq. média (Hz)	Vol. lido médio (L)	Erro médio (%)
Baixa	–	–	–	–	–	–
Média	–	–	–	–	–	–
Alta	–	–	–	–	–	–
Geral		–	–	–	–	–

Fonte: Do autor.

3.3 PRECISÃO VOLUMÉTRICA E REPETIBILIDADE

A Tabela 4 apresenta a precisão na campanha de validação, com o fator teórico e o calibrado, e a Figura 4, a distribuição do erro por repetição.

Tabela 4 - Precisão volumétrica por faixa de vazão na campanha de validação

Faixa	n	Vazão média (L/min)	EA médio (L)	EPM antes (%)	EPM depois (%)	Viés depois (%)	Redução (%)
Baixa	–	–	–	–	–	–	–
Média	–	–	–	–	–	–	–
Alta	–	–	–	–	–	–	–

Fonte: Do autor.

Figura 4 - Erro percentual por repetição em cada faixa de vazão, antes e depois da calibração

Gráfico gerado com pandas/matplotlib a partir do CSV (diagrama de caixa por faixa).

Fonte: Do autor.

A calibração reduziu o EPM de X,XX% para X,XX%, uma redução de XX%. O maior erro ocorreu na faixa de baixa / média / alta vazão (X,XX%) e o menor na de baixa / média / alta (X,XX%). Interpretar: sensores de turbina de baixo custo perdem linearidade perto dos limites da faixa operacional (relacionar com a frequência da Tabela 3); se o viés residual mudar de sinal entre faixas, um único K não corrige toda a faixa, e uma calibração por faixa seria a evolução natural.

A Tabela 5 apresenta a dispersão do volume lido. O CV foi maior na faixa de baixa vazão (X,XX%) e se manteve abaixo / ultrapassou o limite de 5%. Discutir: um sensor pode ser repetível e enviesado; a calibração

corrige o erro sistemático, mas não a dispersão, que inclui a incerteza do recipiente e o tempo de reação ao fechar o registro.

Tabela 5 - Repetibilidade do volume lido por faixa de vazão (fator calibrado)

Faixa	n	Vol. de referência (L)	Média lida (L)	Desvio padrão (L)	CV (%)
Baixa	–	–	–	–	–
Média	–	–	–	–	–
Alta	–	–	–	–	–

Fonte: Do autor.

3.4 COMUNICAÇÃO E ATUALIZAÇÃO

A Tabela 6 apresenta a taxa de entrega. Em rede estável, foi de XX,X%, com N mensagens perdidas em M; com falhas induzidas, de XX,X%. Durante as quedas, as leituras permaneceram na fila do firmware e foram entregues na retomada da conexão, N delas com atraso, o maior de X s, de modo que a interrupção afetou a atualização do painel, mas não o volume registrado. Mascarenhas (2024) relatou quedas de Wi-Fi como limitação; aqui, o contador de sequência permitiu quantificar as perdas, e a fila evitou que as quedas curtas se convertessem em volume não registrado.

Tabela 6 - Taxa de entrega das mensagens

Condição	Esperadas	Recebidas	Duplicadas rejeitadas	Entregues pela fila	Atraso máx. (s)	Taxa de entrega (%)
Rede estável	–	–	–	–	–	–
Falhas de Wi-Fi induzidas	–	–	–	–	–	–

Fonte: Do autor.

A Tabela 7 apresenta a latência em N leituras. A ingestão teve média de X,XX s, atraso pouco significativo, e a parcela dominante foi a exibição, de média X,X s, próxima da metade do intervalo de *polling*, como esperado para uma consulta periódica. A latência total, de média X,X s e percentil 95 de X,X s, é adequada ao acompanhamento do consumo em escala de minutos, frente à leitura mensal do hidrômetro.

Tabela 7 - Latência de atualização por etapa

Etapa	**Média (s)**	**Mediana (s)**	**P95 (s)**	**Máx. (s)**	**DP (s)**
Ingestão (L_{ing})	–	–	–	–	–
Exibição (L_{exib})	–	–	–	–	–
Total (L_{tot})	–	–	–	–	–

Fonte: Do autor.

3.5 AVALIAÇÃO FRENTE AOS CRITÉRIOS DE ACEITAÇÃO

A Tabela 8 confronta os resultados com os critérios definidos na Seção 2.6. Concluir: o sistema atende ao propósito de monitoramento residencial nas faixas em que todos os critérios foram satisfeitos, e a faixa de X vazão deve ser usada com ressalvas. Como a detecção de vazamentos depende mais da persistência do fluxo do que da exatidão do volume, a menor precisão em faixas extremas afeta a quantificação da perda, mas não sua detecção pelo alerta de fluxo contínuo.

Tabela 8 - Resultados frente aos critérios de aceitação

Dimensão	**Métrica**	**Critério**	**Resultado**	**Atende**
Precisão volumétrica	EPM por faixa (depois)	$\leq$ 10%	B / M / A	Sim/Não
Repetibilidade	CV por faixa	$\leq$ 5%	B / M / A	Sim/Não
Comunicação	TE em rede estável	$\geq$ 95%	XX,X%	Sim/Não
Atualização	L_{tot} média	$\leq$ X s	X,X s	Sim/Não

Fonte: Do autor.

3.6 COMPARAÇÃO COM TRABALHOS CORRELATOS

A Tabela 9 situa este trabalho em relação aos correlatos. Diferentemente de Rolle (2024) e Silva (2024b), o sistema não compete em escala nem em sofisticação analítica: dispensa aprendizado de máquina e infraestrutura robusta em favor de uma arquitetura direta, adequada a condomínios geridos sem equipe técnica. Em relação a Barros et al. (2023), a validação abrangeu faixas de vazão representativas do uso residencial, e não apenas condições ideais, e o sistema não se limita a apresentar o consumo: alerta automaticamente o fluxo contínuo característico de vazamentos.

Tabela 9 - Comparação com os trabalhos correlatos

Característica	**Rolle (2024)**	**Silva (2024b)**	**Barros et al. (2023)**	**Medeiros et al. (2021)**	**Mascarenhas (2024)**	**Este trabalho**
Escopo	Rede de distribuição	Rede municipal	Laboratório	Edificação	Residência	Condomínio, por unidade
Hardware	Sensores IoT + servidor	Sensores IoT + servidor	Sensores IoT	NodeMCU ESP8266	Arduino + ESP8266	ESP32
Processamento	Aprendizado de máquina	Inteligência artificial	Agregação simples	Lógica embarcada	Lógica embarcada	Agregação no banco + alerta de fluxo contínuo
Armazenamento e interface	–	Painel de larga escala	–	Planilha Google	Planilha + Power BI	PostgreSQL/ TimescaleDB; app web e móvel
Controle de acesso	–	–	–	Não	Não	JWT, perfis e chave por sensor
Validação	Localização de falhas	Escala de dados	Ambiente controlado	Perda detectada	Fator de correção	Erro por faixa, CV, entrega e latência

Fonte: Do autor.

Em relação a Medeiros et al. (2021) e Mascarenhas (2024), as diferenças estão no software e no contexto: ambos armazenaram os dados em planilhas, sem autenticação ou controle de acesso, e o segundo dependia do Power BI, cuja versão gratuita não permite compartilhar painéis. Este trabalho os substituiu por uma API própria, um banco de série temporal e um aplicativo com visões distintas para síndico e proprietário. Além disso, o ESP32 dispensou o módulo Wi-Fi externo apontado por Mascarenhas (2024) como fonte de instabilidade, e o sistema foi modelado para múltiplas unidades, com métricas de entrega e latência que não foram quantificadas nos correlatos.

4 CONCLUSÃO

Este trabalho apresentou o desenvolvimento e a validação de um sistema de telemetria para o monitoramento contínuo do consumo de água em condomínios, composto por um protótipo com sensor YF-S201 e ESP32, uma API REST em Spring Boot com PostgreSQL e TimescaleDB e uma aplicação web e móvel em React Native com Expo. Os objetivos

foram atingidos: a arquitetura foi modelada em três camadas e para múltiplas unidades; o protótipo coletou e transmitiu as leituras por Wi-Fi; a API e a aplicação armazenaram e apresentaram o consumo e os alertas de fluxo contínuo com controle de acesso por perfil; e os testes de bancada quantificaram o desempenho em quatro dimensões.

Após a calibração, o erro percentual médio foi de X,XX%, X,XX% e X,XX% nas faixas baixa, média e alta, uma redução de XX% em relação ao fator teórico; o coeficiente de variação ficou entre X,XX% e X,XX%; a taxa de entrega foi de XX,X% em rede estável e de XX,X% com falhas induzidas; e a latência média foi de X,X s, dominada pelo *polling*. Síntese: o sistema atende / atende parcialmente ao propósito de monitoramento residencial, com ressalva para a faixa de X vazão.

As contribuições são uma arquitetura IoT de baixo custo e código aberto para a visibilidade do consumo por unidade, um modelo de autorização que reflete a hierarquia entre síndico e proprietários, a detecção automática de fluxo contínuo como alerta de vazamento, um firmware testável sem hardware e tolerante a quedas curtas de rede e uma validação que mensurou, além da precisão, a repetibilidade, a latência e a confiabilidade da transmissão. Como limitações, a validação ocorreu em bancada, com um sensor físico e demais unidades simuladas; a fila de reenvio fica em memória volátil e se perde se o microcontrolador reiniciar; o ESP32 usa HTTPS sem validar o certificado do servidor e, na versão web, o token de acesso fica no armazenamento local do navegador, escolhas aceitáveis em um protótipo, mas não em produção; os alertas são exibidos apenas na aplicação, sem notificação ativa; e o consumo é agregado sob demanda, sem retenção ou compressão. Como trabalhos futuros, recomendam-se a persistência da fila de reenvio em memória não volátil, a validação do certificado do servidor no firmware, a notificação dos alertas por e-mail ou *push* e novos alertas por limiar de volume, agregados contínuos do TimescaleDB, a substituição do *polling* por *Server-Sent Events*, a calibração por faixa de vazão e a validação em um condomínio real, com múltiplas unidades em operação simultânea.

DECLARAÇÃO DE USO DE INTELIGÊNCIA ARTIFICIAL

Os autores declaram que ferramentas de Inteligência Artificial generativa foram utilizadas exclusivamente como apoio auxiliar à elaboração deste trabalho, incluindo assistência na revisão linguística, estruturação textual e organização de ideias. Nenhuma ferramenta de IA foi utilizada

para substituição da autoria intelectual, análise científica, interpretação dos resultados ou tomada de decisões acadêmicas. A responsabilidade integral pelo conteúdo, originalidade, veracidade das informações e conformidade com os princípios de integridade científica permanece integralmente atribuída aos autores.

REFERÊNCIAS

ASHTON, K. **That 'internet of things' thing**. RFID Journal. 2009, Disponível em: <https://www.rfidjournal.com/that-internet-of-things-thing>. Acesso em: 20 mar. 2026.

BARROS, N. N. et al. **Monitoramento por iot para avaliação de consumo de energia e água de ambiente laboratorial**. Ambiente Construído, ANTAC, v. 25. 2023, Disponível em: <https://doi.org/10.1590/s1678-86212025000100843>. Acesso em: 20 mar. 2026.

CLOUDFLARE. **Cloudflare Tunnel**. 2026. Disponível em: <https://developers.cloudflare.com/tunnel/>. Acesso em: 19 out. 2026.

ESPRESSIF SYSTEMS. **ESP32**. 2026. Disponível em: <https://www.espressif.com/en/products/socs/esp32>. Acesso em: 17 set. 2026.

EXPO. **Expo Documentation**. 2026. Disponível em: <https://docs.expo.dev>. Acesso em: 17 set. 2026.

MAGRANI, E. **A Internet das Coisas**. Rio de Janeiro: FGV Editora. Disponível em: <https://repositorio.fgv.br/handle/10438/24135>. 2018. Acesso em: 20 mar. 2026.

MASCARENHAS, D. d. M. **Medidor de Água Inteligente para Monitoramento do Consumo de Água**. Dissertação (Mestrado) — Universidade Federal do Ceará, Campus de Quixadá, Quixadá, 2024. Disponível em: <https://repositorio.ufc.br/handle/riufc/78587>. Acesso em: 17 set. 2026.

MEDEIROS, L. E. L. d. et al. **Desenvolvimento de um protótipo inteligente utilizando sensor turbina para detecção de vazamentos em edificações**. Revista Ibero-Americana de Ciências Ambientais, v. 12, p. 126–134. 2021, Disponível em: <https://sustenere.inf.br/index.php/rica/article/view/6836>. Acesso em: 17 set. 2026.

PAVANELI, T. H. **Desenvolvimento de uma API e uma Aplicação Web para auxiliar na análise de dados providos por Aplicação Móvel**. Dissertação (Mestrado) — Universidade Federal de Uberlândia (UFU), Uberlândia, 2023. Disponível em: <https://repositorio.ufu.br/handle/123456789/37886>. Acesso em: 20 mar. 2026.

PLATFORMIO. **PlatformIO Documentation**. 2026. Disponível em: <https://docs.platformio.org>. Acesso em: 17 set. 2026.

RED HAT. **O que é uma API (Interface de Programação de Aplicações)?** 2023. Disponível em: <https://www.redhat.com/pt-br/topics/api/what-are-application-programming-interfaces>. Acesso em: 20 mar. 2026.

RODRIGUES, F. F. et al. **Estudo de aplicações da internet das coisas em um ambiente acadêmico**. Revista Espacios, v. 35, n. 13. 2014, Disponível em: <http://www.revistaespacios.com/a14v35n13/14351309.html>. Acesso em: 20 mar. 2026.

ROLLE, R. P. **Detecção e localização de vazamentos utilizando Internet das Coisas e Aprendizado de Máquina**. Tese (Doutorado) — Universidade Estadual Paulista (UNESP), Ilha Solteira, 2024. Disponível em: <https://hdl.handle.net/11449/254317>. Acesso em: 20 mar. 2026.

SANEAMENTO HOJE. **Estudo de Perdas de Água 2024 (SNIS, 2022): Desafios na Eficiência do Saneamento Básico no Brasil**. 2024. Disponível em: <https://saneamentohoje.com.br/estudo-de-perdas-de-agua-2024-snis-2022-desafios-na-eficiencia-do-saneamento-basico-no-brasil/>. Acesso em: 20 mar. 2026.

SANTOS, B. P. et al. **Internet das coisas: da teoria à prática**. In: **Anais do XXXIV Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos (SBRC)**. SBC, 2016. 2016. Disponível em: <https://homepages.dcc.ufmg.br/~mmvieira/cc/papers/internet-das-coisas.pdf>. Acesso em: 20 mar. 2026.

SEEED STUDIO. **G1/2 Water Flow Sensor (YF-S201): datasheet**. 2026. Disponível em: <https://wiki.seeedstudio.com/Water-Flow-Sensor/>. Acesso em: 17 set. 2026.

SILVA, A. C. et al. **Gestão inteligente de consumo de água: um estudo sobre a aplicação de tecnologias avançadas de internet das coisas (iot)**. In: ABES. **Anais do 33º Congresso Brasileiro de Engenharia Sanitária e Ambiental**. 2025. 2025. Disponível em: <https://abes-dn.org.br/anaiseletronicos/33cbesa/117_tema_xi.pdf>. Acesso em: 20 mar. 2026.

SILVA, H. S. **Telemetria de processos industriais: uma proposta de sistema de monitoramento de água em linha de produção usando IoT**. Dissertação (Mestrado) — Instituto Federal do Amazonas (IFAM), Manaus, 2024. Disponível em: <https://repositorio.ifam.edu.br/jspui/bitstream/4321/1665/1/TELEMETRIA%20DE%20PROCESSOS%20INDUSTRIAIS_SILVA_2024.pdf>. Acesso em: 20 mar. 2026.

SILVA, J. B. **Sistemas IoT e processamento de dados baseados em inteligência artificial para monitoramento de pressões e vazões na rede de distribuição de água**. Dissertação (Mestrado) — Universidade Federal da Paraíba, João Pessoa, 2024. Disponível em: <https://repositorio.ufpb.br/jspui/handle/123456789/34745>. Acesso em: 20 mar. 2026.

SILVA, M. L. R. **Gestão de recursos hídricos: utilização de internet das coisas (iot) e big data no processo de tomada de decisão**. Sustentare WIPIS. 2021, Disponível em: <https://www.sustentarewipis.com.br/wp-content/uploads/artigos/2021/425164.pdf>. Acesso em: 20 mar. 2026.

SPRING. **Spring Boot Reference Documentation**. 2026. Disponível em: <https://docs.spring.io/spring-boot/>. Acesso em: 17 set. 2026.

TELETIME. **Brasil fecha 2025 com 53 milhões de chips para Internet das Coisas**. 2026. Disponível em: <https://teletime.com.br/02/02/2026/brasil-2025-537-milhoes-iot/>. Acesso em: 20 mar. 2026.

TIMESCALE. **TimescaleDB Documentation**. 2026. Disponível em: <https://docs.timescale.com>. Acesso em: 17 set. 2026.

UNESCO. **Relatório Mundial das Nações Unidas sobre o Desenvolvimento dos Recursos Hídricos 2024: Água para a prosperidade e a paz; resumo executivo**. Paris: UNESCO, 2024. Disponível em: <https://unesdoc.unesco.org/ark:/48223/pf0000388950_por>. Acesso em: 20 mar. 2026.

UNESCO. **Relatório Mundial das Nações Unidas sobre o Desenvolvimento dos Recursos Hídricos 2025: Montanhas e geleiras: torres de água; resumo executivo**. Paris: UNESCO, 2025. Disponível em: <https://unesdoc.unesco.org/ark:/48223/pf0000393071_por>. Acesso em: 20 mar. 2026.

VERCEL. **Vercel Documentation**. 2026. Disponível em: <https://vercel.com/docs>. Acesso em: 19 out. 2026.